

Stock Correlation Prediction

Introduction

The aim of this project is to analyze correlations among stocks listed on the NASDAQ Stock Exchange using graph-based machine learning techniques. By constructing a correlation graph where nodes represent individual stocks and edges represent statistical relationships between them, we will apply graph algorithms to identify communities of related stocks (which may correspond to sectors or cross-sector clusters), uncover inter-sector dependencies, and detect potential systemic risk patterns within the market.

Importance of this task:

The project would play an important role in discovering connections between a basket of stocks which could affect each other during a financial crisis. Therefore an exploration of this project will help to explore and understand methods of financial risk detection and management.

Methodology:

- **Stack:** `yfinance` (prices), `pandas/numpy` (log-returns, rolling windows), `scikit-learn` (models & `TimeSeriesSplit`), `statsmodels` (checks), `sklearn.covariance.LedoitWolf` (shrinkage).
- **Target:** For each pair, compute rolling Pearson correlation over WWW days (e.g., 60/90) and predict **next-window correlation**. Model in **Fisher-z** space and invert at the end.
- **Baselines:** (1) Last value (carry-forward), (2) **EWMA** correlation, (3) **Ledoit-Wolf** shrinked covariance -> correlation.
- **Features:** Last corr, change in corr, each stock's rolling volatility, market beta, same-sector flag, recent co-movement stats (e.g., rolling cosine similarity of returns).
- **Models:** **Ridge/Lasso** (regularized linear) and **XGBoost** (nonlinear).

- **Validation: Walk-forward** (rolling origin) with `TimeSeriesSplit`; evaluate pairwise errors and rebuild the full correlation matrix (make PD via shrinkage).

What baselines we can use to compare our results:

To evaluate the success of this project, the results will be compared against the Global Industry Classification Standard (GICS), which groups companies into sectors and sub-sectors based on their primary business activities. Since companies within the same sector are generally expected to move similarly, this standard serves as an effective baseline for validation.

To compare performance, we will calculate the Mean Squared Error (MSE) between the predicted stock returns (one predicted by our graph-based model and the other estimated from the average return of all stocks within the same GICS sector/sub-sector) and the actual observed returns.

The difference in MSE between the graph-based model and the sector baseline will indicate how effectively the proposed architecture captures real market dynamics.

Expected outcomes:

- **Forecasting:** Lower **RMSE/MAE** vs baselines; higher **directional (sign) accuracy** and better calibration.
- **Stability:** Better-conditioned, **PD** correlation/covariance matrices; reduced estimation noise via shrinkage.
- **Portfolio impact:** Lower realized volatility/drawdowns, higher **Sharpe**, and **lower turnover** when using predicted covariances.
- **Hedging:** Improved hedge ratios -> **lower tracking error** in pairs/sector hedges.
- **Insights:** Clusters that align with sectors; clear **regime sensitivity** (model captures correlation spikes in high-vol periods).
- **Deliverables:** Reproducible code, visuals (heatmaps/graphs), and a short report comparing models vs baselines.

Computing resources:

The computational requirements for this project are moderate and can be fully supported using standard hardware and freely available software tools. All computations, including data retrieval, correlation analysis, and graph construction, will be implemented using efficient Python-based libraries. Cloud computing platforms will be employed where additional processing power or GPU support is beneficial. The following summarises the key computational resources and tools to be utilized:

- **Hardware**

- Standard personal computers with 8–16 GB of RAM for local data processing and preliminary analysis.
- Cloud-based environments (e.g., *Google Colab*, *Kaggle Notebooks*) for scalable execution and GPU-enabled tasks such as graph embedding computations.

- **Software and Libraries**

- **Python 3.x** as the primary programming language.
- **Pandas** and **NumPy** for numerical operations and data preprocessing.
- **NetworkX** for graph construction, analysis, and visualization.
- **Matplotlib** and **Plotly** for graphical representation of results.
- **Node2Vec** or **PyTorch Geometric** for potential graph embedding experiments.

- **Data Sources**

- Historical stock price and market data retrieved via the **Yahoo Finance API** or **WhaleWisdom** datasets.
- Optional use of open-source financial repositories for additional sector classification or metadata.

Overall, the computational load of the proposed study remains manageable and does not necessitate high-performance computing infrastructure. The chosen setup ensures reproducibility, accessibility, and efficiency, making it well-suited for a team-based academic project of this scale.